



Building an Expandable, Modular Cloud OS with Docker and Kubernetes

Introduction

Creating an **expandable and modular cloud OS** means designing a computing environment that isn't tied to a single machine, but can span multiple devices, scale with additional hardware, and run various services in a flexible, isolated way. In this paper, we outline how to build such a system using two Apple computers as a starting point: a 2019 MacBook Pro (Core i9, 32 GB RAM, 1 TB SSD) running the latest macOS, and a 2017 iMac 5K (48 GB RAM, 1 TB Fusion Drive) running macOS Ventura. The goal is to leverage these powerful machines to run a unified **Debian "Trixie"** Linux environment (for our custom AI/OS) on top of the existing OS, complete with a full **MATE desktop** GUI accessible remotely. We will use **Docker containers** for modularity and **Kubernetes** (specifically a lightweight distribution) for orchestration, enabling the system to easily expand as more hardware (such as a Supermicro X9QRi-F+ server planned as a storage/backup node) is added. All components will be interconnected via a high-speed Ethernet network (using a Thunderbolt-to-Ethernet adapter for the MacBook) to ensure reliable and fast communication between nodes.

This design offers a number of benefits. First, it provides a **consistent "cloud desktop"** experience: you can access the same Debian-based desktop and tools from any of your devices, without being limited by the host OS ¹. Second, containerization ensures that each service or component of the OS is isolated and portable, making the system highly modular. Third, Kubernetes allows these containerized components to be distributed across machines and scaled, truly leveraging the combined CPU power (multiple cores of the i9 and i7 CPUs), large memory (32 GB + 48 GB), and storage of both machines. Overall, this approach merges the convenience of a personal OS with the robustness of cloud infrastructure, tailored to your hardware. Below, we detail the architecture and steps to implement this expandable cloud OS.

Hardware and Network Setup

Hardware Overview: Our initial cluster will consist of two Apple machines: the **2019 MacBook Pro** (6-core/8-core Intel Core i9 with 32 GB RAM and 1 TB SSD) and the **2017 iMac 5K** (quad-core Intel CPU with 48 GB RAM and a 1 TB Fusion Drive). These machines provide a strong foundation – the MacBook Pro offers a high-performance CPU and fast SSD, while the iMac contributes a large amount of RAM and additional storage. Both will remain on macOS as the host OS for now, which means we won't directly install Debian on the bare metal but run it on top of macOS (via virtualization or containers). Keeping macOS intact ensures you can still use the native OS, but our cloud OS environment will largely run within Debian instances.

Networking Configuration: To connect the machines in a cluster, we will use a **wired Gigabit Ethernet** network. A direct Ethernet connection (or through a Gigabit switch/router) offers low latency and high bandwidth, important for fast data exchange in the cluster. The MacBook Pro lacks a built-in Ethernet port, so we'll use a Thunderbolt or USB-C to Ethernet adapter to give it a reliable network interface. Once connected, both machines should be on the same LAN segment and can be assigned static IP addresses (or

reserved DHCP leases) to simplify cluster setup. This direct Ethernet link ensures that inter-node communication (such as Kubernetes control signals or data transfers between Docker containers on different hosts) occurs with minimal delay and maximum throughput. It's advisable to verify connectivity (e.g., by pinging between the MacBook and iMac) and to configure any necessary firewall settings on macOS to allow the cluster traffic.

Future Expansion: The hardware setup is designed to be expandable. In the near future, you plan to add a **Supermicro X9QRI-F+ server** to the mix, which will act as a storage and backup hub. That Supermicro is a robust server-class machine (a quad-socket Xeon system) with potential for massive storage (it supports many drives and up to 1 TB of RAM) ² ³. When it comes online, it can either join the existing cluster as another node or operate as a dedicated storage server accessible to the cluster. We'll ensure our architecture can integrate this additional hardware seamlessly – for example, by using network storage protocols or Kubernetes storage classes to make the Supermicro's disks available to the cloud OS. The **single-cluster approach** will be used initially (both the MacBook and iMac forming one Kubernetes cluster), but this approach won't preclude running multiple clusters later if needed (e.g., one for core services and one for experiments, or an on-premises cluster and a cloud cluster, etc.). The key is that the design remains flexible: new nodes can be added to the cluster with minimal reconfiguration, and services can be redistributed to take advantage of new resources.

Base OS: Debian Trixie with MATE Desktop Environment

At the heart of our cloud OS is **Debian “Trixie”**, the codename for Debian 13 (the latest release of Debian as of 2025). We choose Debian for its renowned stability, large software repositories, and strong community support. Trixie, in particular, comes with updated packages and support for modern hardware and architectures ⁴. It also offers multiple desktop environment options – we will use **MATE 1.26** ⁵ as our desktop environment. MATE is a continuation of the classic GNOME 2 interface; it is lightweight, efficient, and well-supported in Debian, making it ideal for a virtual/remote desktop scenario. By using MATE, we ensure the GUI is not too resource-intensive, leaving more CPU and RAM headroom for other services and containers.

Running Debian on Mac Hardware: Since our Macs are running macOS, we won't replace their OS but instead run Debian Trixie in a contained manner. There are two main approaches here: virtualization or containerization. For a full desktop OS experience, the straightforward method is to use **virtualization** – for example, installing a hypervisor like VirtualBox, VMware Fusion, Parallels, or the open-source UTM (QEMU) on macOS, and then creating a Debian Trixie virtual machine. This VM will have the MATE desktop installed and act as one node of our cloud OS. Each physical machine can run its own Debian VM: one on the MacBook Pro and one on the iMac. We'll allocate a reasonable amount of resources to each VM (e.g. 4–8 CPU cores and 16 GB RAM for the Debian VM on the MacBook, and perhaps 8 cores and 32 GB RAM for the VM on the iMac, given its higher RAM, adjusting as needed). This leaves some resources for macOS, but the majority can be devoted to the guest Debian system. By using hardware virtualization, we create an isolated environment where Debian has full control of its kernel and modules, which is useful for running Docker and Kubernetes inside it.

It's also possible to use **containers** to run Debian userland on macOS (since Docker can run Linux containers on Mac by using a lightweight VM under the hood). However, running a full desktop environment purely in Docker on macOS is more complex – it would involve containerizing the X11 server and desktop, and is generally less straightforward than using a VM. Therefore, we will primarily use VMs for anything

requiring a GUI (the “virtual desktop”). In summary, each Mac will host a Debian Trixie instance (preferably via VMs for full OS functionality). The combination of macOS + VM means our environment uses **both** hardware virtualization and containerization in tandem – a technique that provides maximum portability and flexibility ⁶. In our case, macOS provides the base system, the VM provides a Linux kernel environment, and within that we will deploy containers. This layered approach (containers inside VMs) is a common design to get the benefits of both technologies ⁷.

Setting up Debian + MATE: Once the virtualization is prepared, installing Debian Trixie is straightforward using the standard ISO installer. We will include the MATE desktop packages (Debian’s installer or `tasksel` can install “Debian desktop environment – MATE”). After installation, it’s important to configure the VM for **virtual desktop access**. Since these Debian VMs might run headless (especially if the Mac is not actively showing the VM), we want to be able to access the MATE desktop remotely from any device. We have a few options:

- **X2Go:** X2Go is a remote desktop solution that uses NX technology over SSH. It’s highly efficient and works well even over low bandwidth, offering a responsive GUI experience ⁸. X2Go pairs perfectly with MATE (and XFCE/LXDE) – in fact, it’s known to work fine with MATE out-of-the-box (with compositing disabled for performance) ⁹. We can install an X2Go server on the Debian VM and an X2Go client on whichever system we use to access it (e.g. the MacBook’s macOS, or any other PC). This will allow us to launch our cloud desktop and use it interactively, as if we were sitting at a Debian machine, from anywhere. The advantage of X2Go is that it’s secure (using SSH) and can resume sessions, so you could start a session from the iMac and later reconnect from the MacBook, continuing where you left off – giving you a **consistent work environment across multiple devices** ¹.
- **VNC or RDP:** Alternatively, we could configure a VNC server or use RDP (Remote Desktop Protocol via `xrdp`). These solutions also allow remote GUI access. VNC is simpler but can be slower and less secure; RDP (with `xrdp` on Linux) can integrate with Windows/Mac RDP clients. However, X2Go generally outperforms VNC in speed and simplicity (and it supports sound and printing if needed). For our purpose, we’ll likely stick with X2Go for the “cloud OS desktop.”
- **Direct Display:** If you prefer, you could also use the VM’s console directly on the host machine (VirtualBox GUI, etc.) for local access. For instance, the iMac could display the Debian VM on part of its screen. But for the MacBook (which may not always be running the VM visibly) and to allow remote access, X2Go/SSH is preferred.

Once this is set up, we will have Debian Trixie running with a full MATE desktop environment on both machines. This environment constitutes the **AI/OS base** – you can install any Linux software here, use the terminal, run development tools, etc. The “AI/OS” might include special AI software or agents (for example, an AI assistant, machine learning libraries, etc.) – those can be installed in this environment or run as containers as we’ll discuss. Crucially, by having this as a cloud-based desktop, it remains “always on” and accessible. Even if your MacBook is turned off or taken outside, the iMac’s instance could still be running at home, accessible via the internet if configured (using SSH port forwarding or VPN for security). This addresses the use-case of an “*always-online, remotely-accessible*” system ¹⁰. Essentially, your Debian desktop is not tied to one physical screen; it lives in the cluster and you view it from anywhere.

Docker for Modular Components and Services

With the base Debian OS running, the next step is to break down the OS's capabilities into **modular components** using Docker containers. Docker will allow us to package various services or applications of the OS into independent units (containers) that can be deployed and managed more easily than installing everything directly on the OS. This modular design is a key to making the system **flexible and expandable**.

Why Docker? Docker provides operating-system-level virtualization, which is more lightweight than running multiple full VMs. Containers share the host's kernel but isolate the application processes and their dependencies in separate environments. This means we can run many containers on one VM very efficiently – **Docker containers are generally faster and less resource-intensive than full virtual machines** ¹¹. By using containers, we leverage the hardware to the fullest: instead of one giant monolithic system, we can have dozens of micro-services running, each taking only the resources it needs and with minimal overhead.

Modularity through Microservices: We plan to identify the different “features” or services of our AI/OS and encapsulate them in containers. For example, some possible components might be: - **AI services:** If the OS has AI capabilities (say a chatbot, voice assistant, or machine learning model serving), we can run those in containers. For instance, an LLM-based assistant could run as a service listening on an API, packaged in a Docker image with all its ML frameworks installed. This container can be updated or replaced independently of the rest of the system. - **Data storage and databases:** If you need a database (for logging, for the AI's knowledge base, etc.), running it in a container (e.g., a PostgreSQL or MongoDB container) isolates the database and makes it easy to manage its life cycle. The database container can use a persistent volume (possibly on the iMac's disk or the future Supermicro storage) for data durability. - **Web interface or APIs:** Perhaps the AI/OS exposes a web dashboard or mobile UI. That web server can run in its own container (e.g., an Nginx or Node.js container). - **Background jobs or automations:** Any scheduled tasks, file syncing services, or background workers can each be a container. - **Desktop environment integration:** The GUI apps that you run in the MATE desktop could either be installed traditionally or delivered via containers using technologies like Docker AppImage or X11 forwarding from containers. For simplicity, core desktop apps (browser, editor, etc.) can be installed on the Debian VM itself. But anything that benefits from isolation (like trying different versions of a tool) can be containerized.

Using a microservices architecture with Docker means each part of the OS is independently deployable and upgradeable ¹² ¹³. This greatly eases maintenance: for example, you can update the AI model container without disturbing the database or web UI containers. It also enhances reliability and security – if one component crashes or has a vulnerability, it's isolated from others. And importantly, this **modularity** aligns with making the OS *expandable*: you can add new containers (new features) at any time, and if there's more hardware, you can allocate these containers across the machines to balance the load.

Docker Setup: We will install Docker Engine within the Debian Trixie environment on at least one of the nodes (e.g., on both the MacBook's and iMac's Debian VMs). Both nodes should be able to run containers. Because these Debian instances share the same kernel with their VM host only (not with each other or macOS), each will have its own Docker daemon. However, we will unify management through Kubernetes (coming next), which will schedule containers on either node as needed. The Docker installation is straightforward via the official Docker CE packages for Debian, or we could utilize containerd (which is the underlying runtime Docker uses) directly with Kubernetes.

One consideration on macOS: Docker Desktop for Mac could have been an option to run containers, but since we are running Debian in VMs, we'll be running Docker inside the Linux VMs directly (this avoids any Mac-specific limitations and uses standard Linux containerization). As a result, our containers will be running on a Linux kernel (from the Debian VM) which is ideal for compatibility.

It's worth noting that we are effectively combining **virtual machines and containers**: this is not only acceptable, but often recommended for portable, modular deployments ⁷. The VMs give us the correct OS environment (Linux) and isolation from the host, while Docker gives us isolation of applications within that environment.

Kubernetes Cluster Orchestration

With Docker providing the containerization, we introduce **Kubernetes** to handle orchestration – in other words, to manage and coordinate all these containers across our two (eventually more) machines. Kubernetes will treat our collection of hardware as a single unified pool of resources on which it can run containerized workloads. This brings several advantages essential to a cloud OS: automatic rescheduling of services if a node goes down, easy scaling of services (deploying more container instances if load increases), and a declarative way to describe the system's desired state (so everything can be brought up or healed automatically).

Choosing Kubernetes Distribution: A full Kubernetes installation (as used in large data centers) can be quite heavy for a small two-node setup. Fortunately, there are lightweight Kubernetes distributions. We will use **K3s** (by Rancher/SUSE) – a minimal, single-binary Kubernetes that is perfect for this scenario ¹⁴ ¹⁵. K3s is a CNCF-certified Kubernetes, meaning it behaves just like standard Kubernetes but is optimized for resource-constrained or edge environments. It has a tiny footprint (under 100MB binary) and simplifies a lot of the setup (for example, it can use an embedded SQLite datastore by default, instead of a separate etcd cluster for control plane state). Community experiences have shown K3s to be “*ridiculously easy to set up*” and low-maintenance for home labs ¹⁴ – many homelab and self-hosting enthusiasts have successfully moved their services to K3s because of its convenience ¹⁶.

K3s Setup: We will designate one of our Debian nodes as the **Kubernetes master (server)** and the other as a **worker (agent)**. For reliability, the iMac's Debian VM is a good choice for the master, since the iMac can be left powered on as an “always-on” server. The MacBook's Debian VM will then join as a worker node when it's on the network. Installing K3s is as simple as running a script on the master node:

```
curl -sfL https://get.k3s.io | sh -
```

Within about 30 seconds, this will turn that machine into a single-node Kubernetes cluster (server) ¹⁷ ¹⁸. By default, K3s will start the Kubernetes API server, scheduler, controllers, and etc., and even register the node as a worker too (so the master can run workloads) ¹⁹ ²⁰. Next, on the second node (the MacBook's Debian), we run the installer with two environment variables pointing to the master's address and join token. For example ²¹:

```
export K3S_URL="https://<MasterNodeIP>:6443"
export K3S_TOKEN="<the join token from master>"
curl -sfL https://get.k3s.io | sh -
```

This command will install K3s and automatically **join the node to the cluster** as an agent ²² ²³. In a few moments, the MacBook node will appear in the cluster. We can verify on the master with `kubectl get nodes` (K3s includes `kubectl` or we can use an external `kubectl` pointing to the cluster) to see both nodes listed.

K3s Networking and Integration: K3s by default uses an embedded networking layer (flannel) and a default service load balancer/ingress for exposing services. All of this is preconfigured, meaning our two nodes should form a coherent network for containers without extra work. The Debian VMs will each have their own IP on our LAN – K3s will create an overlay network so that containers can communicate across hosts as if on one network. We should ensure that the necessary ports are open between the machines (K3s server uses port 6443 for API, and flannel/agent communication uses UDP ports in the 8472 range for overlays, etc., which on a LAN should be fine by default).

Going forward, we will **deploy applications via Kubernetes** manifest files or Helm charts, rather than manually running `docker run`. For instance, if we want to run a database container, we can write a Kubernetes Deployment for it. Kubernetes will decide which node to run it on (based on resource availability or rules we set). If the MacBook is offline, the master node can run critical services alone (since it's also a worker). When the MacBook comes online, Kubernetes can distribute some load to it – effectively **balancing the use of hardware**.

Cluster State and Storage: In our initial single-master setup, K3s will likely use an embedded SQLite datastore on the master node to store cluster state. This is fine for a small cluster. We should take backups of this state (K3s provides a script to backup the data) especially before making major changes. In the future, when adding the Supermicro server, we might convert to a high-availability setup (multiple masters) for resilience, which would involve using an external datastore like etcd or MySQL for K3s. For now, a single master is acceptable for simplicity.

Using the Cluster: Now we have a functioning Kubernetes cluster spanning the two machines. We will use `kubectl` (the Kubernetes command-line tool) to interact with it. For example, deploying our AI/OS services might involve steps like: - Creating **Namespaces** to group certain applications (perhaps a namespace for core OS services, one for AI services, one for monitoring, etc.). - Deploying our containers as **Deployments** or **StatefulSets** depending on their nature (stateless vs stateful). - Exposing services via **Services** or **Ingress** so that they can be accessed (for example, an ingress to reach a web UI, or a service that the MATE desktop can connect to internally). - Using **Persistent Volume Claims** for any service that needs disk storage (this is where our storage node or iMac's disk comes into play – we might use NFS or hostPath volumes for now to store data, and later migrate to the Supermicro's storage).

Kubernetes will handle ensuring the desired number of container replicas are running. Should one of the machines go down or a container crash, Kubernetes can restart or reschedule those workloads on the available machine automatically, offering a level of self-healing.

Figure: Architecture of a K3s cluster with one server node and multiple agent nodes. In our setup, the iMac's Debian VM serves as the K3s server (control plane + agent), and the MacBook's VM joins as an agent. Kubernetes manages containerized services across both. ²⁴

By adopting Kubernetes, **expansion is straightforward**: when you add the Supermicro server later, you can simply install K3s on it with the appropriate `K3S_URL` and `K3S_TOKEN` as an agent (or possibly as a second master, if aiming for HA). It will join the cluster, and its immense resources (CPU, RAM, disk) become available for scheduling pods. This could allow you, for example, to scale out an AI workload to use all machines or to offload heavy tasks to the powerful server. Kubernetes acts as the “operating system” of your cloud, abstracting the hardware into a unified resource pool.

Leveraging Hardware Capabilities

One of the objectives is to **leverage the best that the hardware has to offer**. Our design ensures that each piece of hardware can be utilized to its strengths within the cluster:

- **CPU and Performance:** Both the MacBook Pro's i9 processor and the iMac's processor contribute computing power. Docker and Kubernetes allow running workloads in parallel across all CPU cores of both machines. For compute-intensive tasks (like AI model inference or data processing), pods can be scheduled on the machine that has free CPU cycles. If an AI service container requires a lot of CPU, Kubernetes might place it on the iMac when the MacBook is away or busy, or vice versa. We could also label nodes and dedicate certain tasks to the more powerful CPU if needed. This parallel usage of processors means the combined cluster can handle more than either machine alone.
- **Memory and Efficiency:** With 32 GB and 48 GB RAM on the two systems, memory is abundant. This is ideal for running memory-hungry applications (e.g. caching services, in-memory databases, or machine learning models loaded into RAM). By containerizing apps, we can run many services without the overhead of launching separate VMs for each – containers share the OS and **can utilize memory more efficiently** than multiple full OS instances would ¹¹. For example, the base Debian OS might use a couple of gigabytes, leaving tens of GB free for containers. If one node's RAM starts filling up, we can deploy new containers to the other node to balance the usage.
- **Storage and I/O:** The MacBook's 1 TB SSD is very fast, great for databases or I/O intensive tasks. The iMac's 1 TB Fusion Drive is slower for random I/O, but still offers capacity. We might designate the MacBook's storage for performance-sensitive workloads (like a database PV) and use the iMac's storage for less I/O intensive or backup purposes. In Kubernetes, this can be managed by using node selectors or affinity rules for certain pods, or by setting up storage classes. When the Supermicro with (presumably) many drives comes in, we will integrate it as a central storage provider – for example, by setting it up to run a NAS software or exporting NFS shares to the cluster. We could then mount those shares in the Debian VMs or use a Kubernetes NFS-client provisioner to let pods claim space on the NAS. This would effectively **centralize the storage** so that data is accessible regardless of which node a container runs on. Backup routines can be offloaded to the Supermicro as well.
- **Networking:** By using a direct Ethernet connection, we maximize network throughput between containers on different hosts. If we run, say, a distributed database cluster or a web server on one

node and its backend on another, the gigabit (or higher, if using adapters that support 10GbE in future) link ensures they communicate swiftly. Also, for accessing the cloud OS from the outside, we can take advantage of the network. For instance, if you want to access your Debian/MATE desktop remotely (from the internet), you could expose the X2Go or RDP service via a secure ingress or tunnel. Likewise, any web services can be exposed through the iMac's network (which could have a DNS name pointed to it). The cluster can handle ingress routing to direct traffic to whichever node currently runs the service.

- **GPU and Special Hardware:** If either Mac has a notable GPU (some 2019 MacBook Pros and 2017 iMacs have Radeon GPUs), leveraging them is tricky in this setup because macOS and standard VMs don't easily give GPU access to Linux. If GPU computing is desired for AI (like running CUDA workloads), one might consider using an external GPU (eGPU) or eventually the GPUs in the Supermicro (if any) directly on a Linux install. For now, we assume CPU-based computing. However, it's good to know that the architecture could be adjusted to allow GPU pass-through with the right hypervisor or by running Debian directly on hardware later for those needs.
- **Full OS Features:** The user mentioned "all features needed" – implying the cloud OS should feel like a complete OS, not a limited container set. Thanks to Debian + MATE, we indeed have a full OS: you can run graphical applications, browse the web, play media, etc., by connecting to the virtual desktop. Features like sound can be enabled over X2Go (it can forward sound), and USB devices can even be passed into VMs if needed (VirtualBox and others allow USB passthrough, so you could connect a device to the Mac and have Debian see it). Printing and file transfer are also possible (X2Go can share clipboard and folders). So essentially, nothing stops this environment from providing the same features as a normal desktop OS – except now it's running in a cloud-like setup. Users benefit from scenarios like running long tasks on the iMac VM without tying up their MacBook, as DigitalOcean's guide noted: your local device remains free while the server does the heavy work

25 .

In summary, by carefully distributing workloads and using the appropriate tools (containers for efficiency, cluster scheduling for load balancing, remote desktop for UI), we ensure each hardware component is utilized optimally. The design avoids idle resources: if one machine isn't being actively used, it can still run background containers for the cloud OS. And when both are in use, they collaboratively share the load.

Deploying AI/OS Components and Services

Now that the infrastructure is in place, we focus on the actual **AI/OS features** and how to implement them on this platform. The term "AI/OS" suggests the operating system has integrated AI capabilities or is aimed at AI development and usage. Below are some examples of features one might include and how they'd be deployed:

- **AI Assistant / Chatbot:** Suppose the OS has an AI assistant service (for example, a natural language chatbot that can answer questions or control aspects of the OS). We can implement this by running an AI model server (maybe something like an instance of GPT-J or a smaller LLM, or a service calling an external AI API) in a container. If it's open-source, we might containerize the model runtime (with libraries like PyTorch, TensorFlow, etc. in the image). This container could expose an API (e.g., a REST or gRPC endpoint). The assistant front-end (perhaps a GUI app or a terminal client on the MATE desktop) would communicate with this API. By containerizing, we ensure the heavy ML libraries and

dependencies are isolated. If we upgrade the model or switch to a different framework, we just deploy a new container version. The cluster can allocate this container to run on the machine with more free GPU/CPU as needed.

- **Voice Recognition or Personal Assistant Daemon:** If voice control is a feature, a service like **DeepSpeech** or another speech-to-text engine could run in a container, possibly on the machine with a microphone attached (or on the server if audio streams are forwarded). Kubernetes can ensure it runs where it can access the audio input (through device plugins or by routing audio over the network).
- **Data Storage and Knowledge Base:** An AI/OS might need to store data like notes, vector embeddings for semantic search, etc. A database container (SQL or NoSQL) can serve this need. We would deploy, say, a PostgreSQL container, with a persistent volume mount. Because we intend the Supermicro to be a storage hub, we might not worry about huge data until that is set up; once available, we could relocate the database storage to that machine (e.g., via an NFS volume). For now, using the iMac's disk for persistent data is fine. As a precaution, we should schedule periodic backups of important volumes (perhaps by using a backup container or external backup to the Supermicro).
- **Web Interface:** It could be useful to have a web-based interface to some OS functions (especially if accessing from devices where you can't use X2Go easily, like a phone). For example, a web dashboard that shows system status, allows issuing commands, or chatting with the AI assistant. This can be hosted with a lightweight web server container. We might run an Nginx reverse proxy as well to route external requests to the appropriate service (Kubernetes ingress can also do this, but sometimes a dedicated reverse proxy with authentication is handy for a personal setup).
- **Desktop Applications:** Most regular desktop applications (LibreOffice, GIMP, etc.) can be installed directly in the Debian MATE VMs for use when connected via remote desktop. However, to keep the system modular, one could also use containerized applications with GUI, delivered via something like X11 forwarding or VNC. For instance, Docker images exist for browsers or IDEs. In practice, given that we have a full VM anyway, installing these apps normally might be simpler. The modularity focus is more on backend services and AI components.
- **Supporting Services:** We can enhance the OS with additional services:
 - Monitoring (e.g., **Prometheus/Grafana** containers) to keep an eye on resource usage across the cluster.
 - Automation tools (like **Home Assistant** or Node-RED if integrating with home automation).
 - Dev tools like a Git server or CI runner in containers, if the environment is used for development.
 - File synchronization or cloud storage: Since a cloud OS implies access to files from anywhere, we could run something like **Nextcloud** in a container to provide a personal cloud storage solution. The Supermicro's large storage could back this. Users could then access files via WebDAV/Nextcloud clients, effectively creating a personal Dropbox-like service as part of the OS.

Because Kubernetes is in charge, deploying these is uniform: each new feature is defined in YAML and launched into the cluster. The cluster ensures they find a place to run. For example, we might set the AI inference service to have an *affinity* for the iMac node (with more RAM), and maybe the web server can run

on the MacBook node, etc. If one node gets overloaded, we can shift workloads – say, run the database on the Supermicro when it's added, to free up Mac resources for interactive use.

Security is another aspect to consider: with multiple networked components, we should enforce safe configurations. Luckily, within the cluster, Kubernetes can limit which pods can talk to which (NetworkPolicies) and all communications can be kept behind the LAN firewall. For external access, using SSH tunneling or VPN into the network is recommended rather than exposing services directly to the internet, unless we set up proper HTTPS and authentication. For example, to access X2Go from outside, one could SSH into the iMac and forward X2Go's port, or use a VPN into the home network.

Expansion and Future Integration

The architecture we built is inherently **expandable**. When you acquire the **Supermicro X9QRi-F+ server**, integrating it can enhance both performance and storage: - **Adding to Cluster:** The Supermicro can be installed with a Linux OS (Debian Trixie as well, or perhaps a server-oriented distro) directly on the metal for maximum performance. We could then install K3s agent on it, just like we did on the MacBook node. Once joined, it becomes another node in the Kubernetes cluster. You might choose to have it run as a worker node only, or also host an additional master (for high availability of the control plane). In a K3s HA setup, you'd run an external database (perhaps the Supermicro could host a MySQL or etcd for K3s HA) and join two or more masters. But if the iMac remains master and the Supermicro just a worker, that's acceptable too, given a homelab environment. - **Role of the Storage Hub:** With potentially dozens of terabytes of disk capacity, the Supermicro can act as a **NAS or storage node**. One approach is to set up a distributed storage system like **Longhorn** or **Ceph** on the cluster, which would use the Supermicro's disks to provide persistent volumes to Kubernetes ²⁶ ¹⁸ . This way, any container on any node can request storage and the data would reside on the Supermicro (with network access). Alternatively, simpler: run NFS server on the Supermicro and use it to host volumes (Kubernetes can then use PersistentVolume definitions that refer to NFS shares). In both cases, the idea is to centralize data so that if, for example, your AI database runs on the MacBook today and tomorrow on the Supermicro, it's accessing the same underlying storage. - **Offloading Work:** The Supermicro, being a multi-CPU powerhouse, could take on the heavy lifting. You might migrate the AI model serving to that machine (more CPU cores for faster inference, or even add a dedicated GPU to it for AI acceleration). It could also run backups of all data (to an external drive or cloud). Kubernetes can be instructed via taints/tolerations to dedicate certain workloads to the Supermicro (for instance, mark it as a "storage" or "batch jobs" node).

The cluster can be expanded beyond this as well. If you later add more nodes (maybe some Raspberry Pi nodes, or another spare PC), as long as they run a compatible Kubernetes agent, they can join. This is a unified cloud OS – not confined like a traditional OS to one machine, but living on an **elastic cluster**.

We should mention that if the cluster grows significantly, monitoring and management become important. Tools like Kubernetes Dashboard or Lens can help visualize the cluster. Also, consider setting resource requests/limits on containers to avoid one container hogging a whole machine unintentionally.

Finally, **multi-cluster** vs **single-cluster**: The user had indicated interest in "both" eventually – possibly running more than one cluster. With our current design, one cluster is sufficient to span all hardware. However, there are scenarios for multiple clusters: for example, a separate cluster for experimenting with different configurations (maybe using k3d or kind to spin up a test cluster inside), or a second cluster in a cloud provider for off-site tasks that integrates with the home cluster. Kubernetes Federation or simply

using separate kubeconfigs would allow managing them. The modular nature we've built means you could move certain workloads to another cluster if needed (since they're just containers defined by code). But to keep things simple, you might only go multi-cluster if necessary, and otherwise just enlarge the single cluster.

Conclusion

In this paper, we presented a comprehensive plan for constructing a personal **expandable, modular cloud OS** using existing hardware and modern container orchestration tools. By using Debian Trixie with the MATE desktop, we retain a full-featured Linux OS environment (complete with GUI) that can be accessed from anywhere, fulfilling the vision of a “cloud desktop” that is always available ⁸. Docker containers are employed to break down the system into modular pieces, allowing each service or feature to be developed, updated, and scaled independently – an approach that maximizes the efficient use of the powerful MacBook Pro and iMac resources while maintaining flexibility ⁶ ¹¹. Kubernetes (via K3s) ties everything together, turning the two machines into a unified cluster that can seamlessly grow by adding new nodes ¹⁵. The result is an environment where hardware boundaries blur: it doesn't matter which physical machine you're using; you interact with the AI/OS and its applications uniformly, and the system automatically allocates tasks to the available hardware.

This design leverages each component optimally – from CPU multi-threading to large RAM usage and high-speed networking – and is prepared for future enhancements like the integration of a dedicated storage server. It is, essentially, your own private cloud. By following the steps and considerations outlined (from setting up Debian VMs and X2Go remote desktops, to deploying microservices and configuring K3s), one can build an advanced setup that would traditionally be found in enterprise environments, but at home. Such a cloud OS not only provides a rich computing experience (with all features of a normal OS) but also resilience and scalability: if one node is busy or down, the services persist on another; as demands increase, new services can be added or scaled out easily.

In implementing this, one will gain valuable experience in modern infrastructure technology, and also the practical benefit of a powerful, centralized computing platform for personal or development use. Remember to keep security in mind (use SSH tunnels, firewall rules, and monitor your services), and maintain the system (apply updates to Debian, Docker, and Kubernetes regularly, and check that Kubernetes certificates are renewed to avoid any expiry issues as noted by others running home clusters). With care and tweaking, this expandable cloud OS will serve as a robust foundation for any projects – especially AI-related – you undertake, combining the convenience of a traditional OS with the “*always-on, remotely-accessible*” power of the cloud ¹⁰.

Sources:

1. Alexandru Andrei, Matt Abrams. “How To Set Up a Remote Desktop with X2Go on Debian 10.” DigitalOcean, 27 May 2020. – Explains the concept of a “cloud desktop” and using X2Go for always-online remote GUI access ⁸ ¹.
2. UpGuard. “Docker vs VMware: How Do They Stack Up?” – Discusses the differences between containerization and full virtualization, noting that Docker containers are more lightweight than VMs and ideal for modular, portable software deployments ¹¹ ⁶.

3. Hacker News discussion. *"Converting an old MacBook into an always-on personal Kubernetes cluster."* (March 2020) – Community members recommend K3s for home clusters due to its light weight and ease of setup ¹⁴ and share positive experiences running self-hosted services on K3s ¹⁶.
4. K3s Official Documentation – K3s is a simplified, single-binary Kubernetes distribution designed for resource-constrained environments ¹⁵. Its installation and architecture (server/agent model) make it easy to add nodes to our cluster with a one-line command ²² ²³.
5. LinkedIn Post by Reza Bojnordi (2025) – Highlights new features in Debian 13 "Trixie," including updated desktop environments (MATE 1.26) and improved cloud support ²⁷, confirming the use of MATE on Debian Trixie for a modern desktop experience.

¹ ⁸ ⁹ ¹⁰ ²⁵ How To Set Up a Remote Desktop with X2Go on Debian 10 | DigitalOcean
<https://www.digitalocean.com/community/tutorials/how-to-set-up-a-remote-desktop-with-x2go-on-debian-10>

² Supermicro X9QRI-F+ Motherboard - Amazon.com
<https://www.amazon.com/Supermicro-MBD-X9QRI-F-B-X9QRI-F-Motherboard/dp/B00B2W81LK>

³ Supermicro MBD-X9QRI-F+ Quad Socket R(LGA 2011) 6x ... - eBiz Pc
https://www.ebizpc.com/Supermicro-MBD-X9QRI-F-Quad-Socket-R-LGA-2011-p/mbd-x9qri-fplus.htm?srsId=AfmBOoqfhZF0ijj2QVMN4JjgBuRsQOzm2WrgkGp_It9igrhGefF3ZVOe

⁴ ⁵ ²⁷ Debian 13 "Trixie" released with new features | Reza Bojnordi posted on the topic | LinkedIn
https://www.linkedin.com/posts/rezabojnordi_debian-linux-debian13-activity-7327339449548206080-ivQT

⁶ ⁷ ¹¹ ¹² ¹³ Docker vs VMWare: How Do They Stack Up? | UpGuard
<https://www.upguard.com/blog/docker-vs-vmware-how-do-they-stack-up>

¹⁴ ¹⁶ Converting an old MacBook into an always-on personal Kubernetes cluster | Hacker News
<https://news.ycombinator.com/item?id=22661029>

¹⁵ ¹⁷ ¹⁸ ²⁶ K3s
<https://k3s.io/>

¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ What is K3s? Architecture, Setup, and Security - Aqua
<https://www.aquasec.com/cloud-native-academy/kubernetes-101/k3s/>